Escaping Degenerate Functional Loops: Learning Programmatic Action Recovery for Tool-Using LLMs

Anonymous submission

Abstract

Tool-using large language models (LLMs) extend text generation to multi-step interaction with APIs, databases, and external services, where reliable execution requires every tool call to translate into task progress. In this paper, we find that this requirement is frequently violated: after a task is blocked, a tool-using LLM may keep issuing different tool calls whose observations repeat the same unresolved effect on the visible task state. We refer to such episodes as *Functional Loops*, and our analysis on three benchmarks and four LLMs shows that they are prevalent, coincide with lower task performance, and are not well captured by string-level tool repetition. To escape a Functional Loop through a grounded local correction, we propose **LoopBreaker**, a local recovery layer that reuses concrete training-time failures at inference time. During training, LoopBreaker compiles recurring failure patterns into typed Recovery Cards, each pairing a state-sensitive trigger with a grounded and bounded action program. At inference time, a frozen controller admits at most one eligible Card once a Functional Loop is detected, and then returns control to the base LLM. Experiments across three tool-use benchmarks and nine LLMs show that LoopBreaker consistently improves the primary task outcome, and reduces tool calls on Functional Loop-exposed trajectories[1].

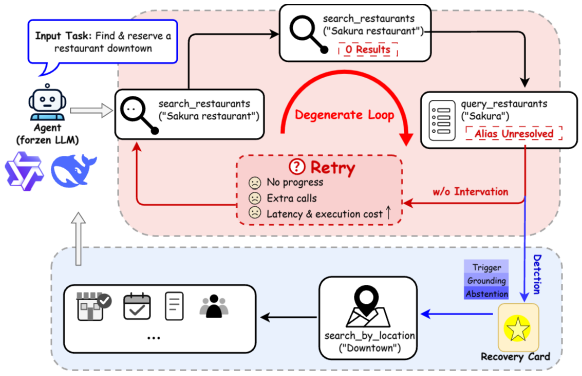


Figure 1: An illustrative functional loop and a local escape by LoopBreaker. Different tool calls repeat the same unresolved effect after feedback is visible, and LoopBreaker admits a grounded location-search Card, applies one bounded correction, and returns control to the base LLM.

Introduction

Tool-using large language models (LLMs) (Yao et al. 2023; Schick et al. 2023; Qin et al. 2024) extend LLMs from text generation to multi-step interaction with APIs, databases, and external services, showing strong performance on a growing range of real-world tool-use tasks (Yao et al. 2025; Barres et al. 2026; Trivedi et al. 2024; Lu et al. 2025a; Chen et al. 2025; He et al. 2026). Reliable execution demands more than selecting a syntactically valid call, since each interaction step should translate into progress on the underlying task.

However, we find that a tool-using LLM often expends interaction budget without producing progress. After a task is blocked, the LLM may keep issuing different tool calls whose observations repeat the same unresolved effect on the visible task state, and every additional call consumes a step without moving the task closer to completion. For instance, in a STATE-Bench Shopping task with a hard $100 budget, the LLM can spend six tool calls searching, retrieving details of an over-budget item, and querying promotions, and none of them repairs the budget violation. We refer to such an episode as a *Functional Loop*, and our analysis across three benchmarks and four LLMs indicates that it is prevalent, is associated with lower task performance, and is not well captured by string-level tool repetition (Section).

Prior work approaches related failure modes from three directions. Early-exit methods reduce ineffective behavior by deciding when to halt a running trajectory (Lu et al. 2025b), while termination alone does not repair a state that a small correction could still resolve. Test-time replanning proposes a recovery plan through another generation step (Kim et al. 2026; Shinn et al. 2023), while the recovery action is only as reliable as the underlying model and adds an unbounded generation cost. Policy optimization treats execution errors as training signals and updates the model with reinforcement learning (Zhang et al. 2026c; Kang et al. 2026; Zhang et al.

[1]Code and data are available in supplementary materials.

2026a; Ta, Zhu, and Shayandeh 2026), and requires gradient access that is unavailable for many production LLMs. Overall, these approaches either halt too early, act without a grounded correction, or rely on write access to model parameters, and none of them treats a Functional Loop as a distinct recoverable execution state.

In this paper, we aim to escape a Functional Loop through a grounded local correction, while leaving the base LLM's policy unchanged. To this end, we begin by asking a foundational question: *when a Functional Loop is detected, what form of correction can be admitted conservatively?* A useful correction should be tied to a specific failure signature, executable in the environment, and admissible only when its trigger and grounding conditions are satisfied. Along with this idea, we propose **LoopBreaker**, a local recovery layer that reuses concrete training-time failures at inference time. As shown in Figure 5, LoopBreaker mines recurring failure patterns in training trajectories offline and compiles each pattern into a typed Recovery Card that pairs a state-sensitive trigger with a grounded and bounded action program. At inference time, a frozen controller admits at most one eligible Card once a Functional Loop is detected, and immediately returns control to the base LLM.

In summary, our key contributions are three-fold.

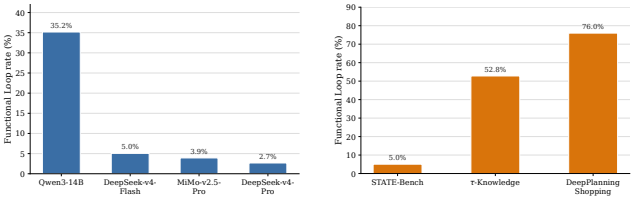
- **Functional Loop Analysis:** To the best of our knowledge, this is the first systematic study of Functional Loops in tool-using LLMs. We formalize a Functional Loop as a feedback-visible sequence that makes no task progress and repeats the same unresolved effect, and characterize its prevalence and impact across four LLMs and three benchmarks.
- **LoopBreaker:** We propose LoopBreaker, which pairs an offline experience compiler with an online selective controller, and turns concrete training failures into grounded and bounded recovery programs at inference time.
- **Strong Performance:** Experiments across three tool-use benchmarks (Lu et al. 2025a; Zhang et al. 2026b; Shi et al. 2026) and nine LLMs show that LoopBreaker consistently improves the primary task outcome, with particularly strong gains on Functional Loop-exposed trajectories.

Functional Loops in Tool-Using LLMs

Functional Loops describe a class of tool-use episodes that consume interaction budget without moving the task forward. We formalize the concept below, and then show three properties that make it a well-defined target for recovery: it is prevalent across models and environments, it degrades task performance, and it is not captured by string-level tool repetition.

Definition. For a tool-use trajectory $\tau = (s_0, a_0, o_0, \ldots, s_T)$, we summarize the visible task state at step t as $q_t = (E_t, S_t, U_t)$, with E_t the non-redundant evidence, S_t the task-relevant environment state, and U_t the set of unresolved requirements. The functional effect of a_t is $e_t = (\Delta E_t, \Delta S_t, \Delta U_t, f_t)$, with f_t a failure signature and $\kappa(e_t)$ a canonicalization for comparison.

Let $p_t = 1$ when a_t either adds new evidence, moves state toward an unresolved requirement, or produces a new failure signature. To make the definition applicable at inference time,



(a) Varying model on STATE-Bench.

(b) Varying benchmark under DS-v4-Flash.

Figure 2: Functional Loops are prevalent among failed trajectories. (a) With STATE-Bench held fixed, the Functional Loop rate varies from 2.7% to 35.2% across four tool-using LLMs. (b) With DeepSeek-v4-Flash held fixed, the rate rises to 76.0% on DeepPlanning Shopping and 52.8% on τ-Knowledge, far above STATE-Bench.

we evaluate progress and repetition over the trailing history $H_t = \{t-w, \ldots, t-1\}$:

$$N(t) = \mathbb{I}\left[U_t \neq \varnothing \ \wedge \sum_{j \in H_t} p_j = 0\right], \qquad (1)$$
$$R_e(t) = \mathbb{I}\left[\substack{\exists i<j,\ i,j \in H_t: \\ \kappa(e_i)=\kappa(e_j) \ \wedge \ o_i \prec a_j}\right],$$

where $o_i \prec a_j$ means o_i was visible before a_j was proposed. A Functional Loop at step t is then

$$\mathcal{L}(t) = N(t) \wedge R_e(t), \qquad (2)$$

and a trajectory is *Functional Loop-exposed* if it contains such an episode. Tool identity is irrelevant: a Functional Loop is defined at the level of functional effects.

Functional Loops are Prevalent Across Models and Environments. We first ask how common such loops are among failed trajectories, and whether their prevalence depends on the base model or on the surrounding environment. Figure 2 separates the two sources of variation. Under the same failed STATE-Bench tasks, Qwen3-14B enters a functional loop in 35.2% of its failed trajectories, whereas DeepSeek-v4-Flash, MiMo-v2.5-Pro, and DeepSeek-v4-Pro loop far less often. With DeepSeek-v4-Flash held fixed, prevalence rises to 76.0% on DeepPlanning Shopping and 52.8% on τ-Knowledge, well above the 5.1% observed on STATE-Bench. Functional loops therefore reflect an interaction between the model's tool policy and the environment, and are especially frequent on long-horizon planning tasks.

Functional Loops Coincide with Lower Task Performance. We next test whether the presence of a functional loop is associated with degraded task performance. Figure 3 compares STATE-Bench trajectories with and without a functional loop. Exposed trajectories score lower for every evaluated model and use 1.6–$2.6\times$ more tool calls per trajectory. The pattern is observational, and difficult tasks may both invite longer interaction and fail more often. Even so, the association identifies a costly execution state that consumes interaction budget without changing the state that blocks completion.

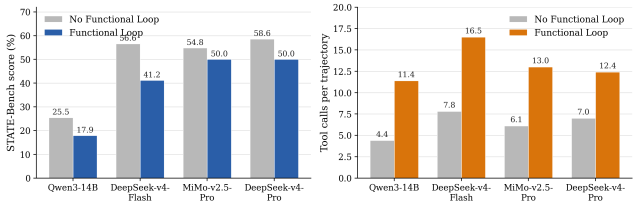


Figure 3: Functional Loop-exposed STATE-Bench trajectories obtain lower scores and consume 1.6–2.6× more tool calls than unexposed ones for every evaluated model.

Step	Tool Action	Feedback	Functional Effect
1–2	`search_products` ($\leq$ \$100)	No matching headphones	No feasible item
3	`search_products` ($\leq$ \$1000)	Finds a \$110 candidate	Budget violated
4	`get_product_details`	Confirms the \$110 price	Violation persists
5–6	Account and promotion queries	No benefit or discount	Constraint unresolved

Table 1: A cross-tool Functional Loop from STATE-Bench Shopping. Steps 4–6 form the Functional Loop: different tools preserve the same unresolved budget violation once the over-budget candidate has been confirmed.

Table 1 traces a representative baseline STATE-Bench Shopping trajectory in which DeepSeek-v4-Flash searches for wireless headphones under a hard \$100 budget. The first two searches produce a new failure signature, and the third search reveals a \$110 item; neither counts as no-progress under our definition. The functional loop begins from Step 4, after the over-budget candidate is confirmed: subsequent queries against the account and promotions repeat the same unresolved budget effect. A functional loop is therefore a signal to consider recovery, and does not by itself imply that the trajectory should be terminated.

Tool-Name Repetition is a Weak Proxy. An obvious alternative is to identify a stalled state by inspecting consecutive tool names. Figure 4 groups STATE-Bench trajectories by whether the same tool is called consecutively. Score differences between the two groups are small and even change direction across models, indicating that string-level repetition is a weak signal for the underlying execution problem. Two different tools can produce the same functional effect on the visible task state, and a second call to the same tool can still produce new evidence. Reasoning at the level of functional effects therefore provides a more useful boundary for recovery.

LoopBreaker

LoopBreaker is a local recovery layer for tool-using LLMs. Its central idea is to *reuse concrete training-time failures* as inference-time experience: once a Functional Loop is detected, a validated local program repairs it without a fresh generation step, so the base LLM is left unchanged. As shown

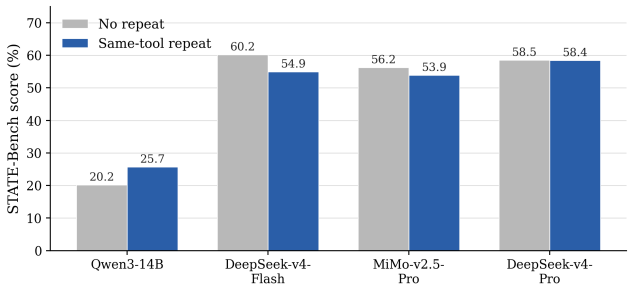


Figure 4: Consecutive same-tool calls are a weak proxy for unresolved effect: STATE-Bench scores are close between the two groups, and the sign of the difference varies across models.

in Figure 5, LoopBreaker realizes this idea in two stages. During training, recurring failure patterns are compiled from trajectories into typed Recovery Cards. At inference time, a frozen controller admits at most one eligible Card at a matched Functional Loop. Each Card is a typed local program with an executable body, grounded arguments, and a bounded budget, and admission is deterministic. When the trigger, grounding, or validity check fails, LoopBreaker preserves the baseline action.

Offline Compilation

Mine Failure Patterns. The upper path of Figure 5 starts from one benchmark's training trajectories. Failed prefixes form a noisier signal than successful trajectories, since most of them do not admit a unique repair. LoopBreaker therefore admits only prefixes that satisfy Eq. 2, and among these it retains a candidate only if the failure pairs with an executable and grounded replacement. Concretely, at each pre-action prefix, N recalls a no-progress state and $\kappa(e)$ tests whether feedback-visible calls repeat the same unresolved effect. We represent a candidate as

$$x_i = (s_i, a_i, e_i, \Delta E_i, \Delta S_i, u_i, \hat{a}_i, g_i, r_i)\,, \tag{3}$$

where $\hat{a}_i$ is an executable replacement, g_i is its visible grounding source, and r_i is a risk tag. A replacement may repair an argument or a blocked prerequisite, and cannot invent entities or relax user constraints. Candidates are then grouped by unresolved effect, action transition, and grounding source.

Compile and Validate Cards. Each supported group is compiled into a typed Recovery Card

$$c = (z_c, \phi_c, \psi_c, \rho_c, g_c, b_c, r_c), \tag{4}$$

where z_c identifies the failure family, ϕ_c the trigger, ψ_c the abstention guards, ρ_c the executable program, g_c the grounding rule, b_c the budget, and r_c the risk tag. A Card is therefore a typed local program with executable arguments, and its trigger and effect are both verifiable in the environment. When snapshots are available, baseline and Card actions are compared from the same state; otherwise we use paired execution without claiming same-state evidence. Retention is gated by task outcome and execution validity, while loop and cost measures serve as diagnostics.

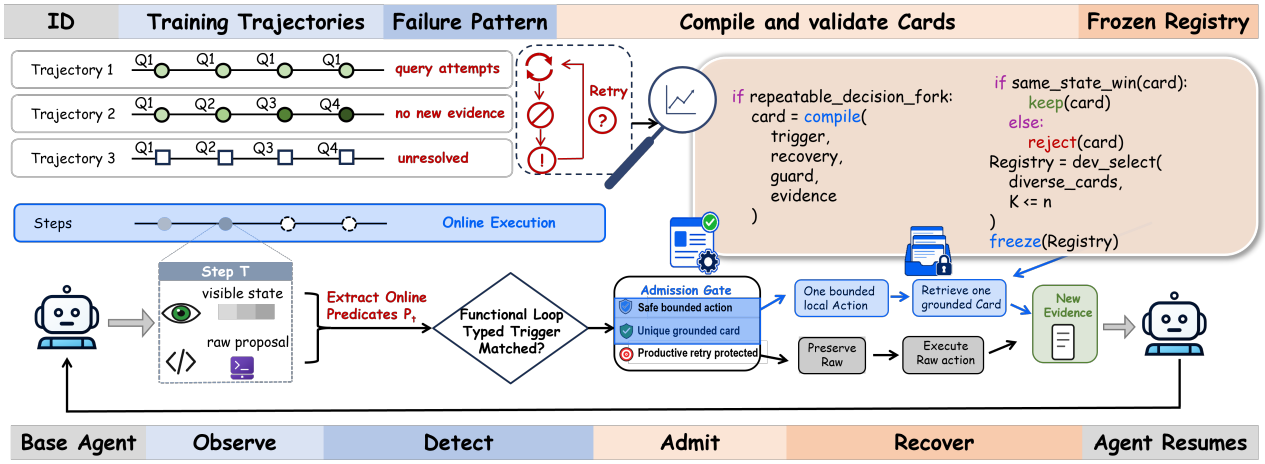


Figure 5: Overview of LoopBreaker. The offline path mines recurring failure patterns in training trajectories, compiles each pattern into a typed Recovery Card, and stores validated Cards in a frozen benchmark-specific Registry. The online path detects a Functional Loop, admits at most one eligible Card, executes the resulting action in the environment, and returns control to the base LLM.

Freeze the Registry. We reject candidates that rely on future or test-time information, ambiguous entities, or constraint violations. Every Card definition together with the controller's runtime settings is frozen before held-out testing. Training-time induction is protocol-constrained: a coding LLM may normalize training failures and instantiate the fixed Card schema, and researchers set the retention protocol without accessing test tasks or outcomes.

Online Recovery

Observe and Detect. In the lower path of Figure 5, the LLM receives tool feedback and proposes a baseline action a_t. Observe exposes the visible history and state s_t, and Detect applies Eq. 2: $N = 1$ recalls no progress, and $R_e = 1$ confirms a repeated feedback-visible effect. Detection therefore reasons at the level of *functional effects*, so a surface tool-name repeat alone is not sufficient to trigger recovery. Detection alone does not authorize intervention.

Admit. The controller then retrieves Cards whose trigger matches the current functional loop family. The eligible set at step t is

$$\mathcal{C}_t = \{c \in \mathcal{R} : \phi_c(s_t, a_t) = 1 \wedge \psi_c(s_t, a_t) = 0 \wedge b_c > 0, \\ \mathrm{Unique}(g_c(s_t)) = 1 \wedge \mathrm{Valid}(\rho_c(s_t, a_t)) = 1\}, \tag{5}$$

where ϕ_c matches the state and effect, ψ_c protects productive retries, and b_c is the remaining budget. Admission also requires unique grounding and a schema-valid, constraint-preserving action. A frozen priority order favors smaller changes and stronger grounding. When recent progress, ambiguous grounding, or a failed validity check is present, LoopBreaker preserves the baseline proposal. This selective admission is what keeps a mined Card from harming a productive retry, and its practical value is quantified in Section .

Recover and Resume. For the highest-priority admitted Card $c^\star$, the executed policy is

$$\pi_{\mathrm{LB}}(s_t, a_t) = \begin{cases} \rho_{c^\star}(s_t, a_t), & \mathcal{C}_t \neq \varnothing, \\ a_t, & \mathcal{C}_t = \varnothing. \end{cases} \tag{6}$$

The environment executes the admitted Card program or the baseline action, and control returns to the LLM. Cards cannot invoke one another without an intervening LLM proposal, so LoopBreaker keeps the base LLM in charge of the trajectory.

Experiments

We compare baseline tool-using LLMs against the same models equipped with LoopBreaker. Official task measures are primary, and functional-loop behavior and tool cost are reported as execution diagnostics.

Benchmarks

STATE-Bench is a stateful conversational tool-use benchmark. STATE-Bench v0.8.1 contains 50 held-out tasks in each of Travel, Customer Support, and Shopping Assistant. Following the official protocol, Pass@1 is the average completion rate over five runs per task, Pass^5 is the fraction of tasks completed in all five runs, and UX averages the user-experience score on valid judged trajectories.

DeepPlanning (Zhang et al. 2026b) is a benchmark for long-horizon agentic planning under verifiable constraints. We use its 120-task Shopping Planning split, and report Match Score and Case Accuracy.

τ-Knowledge (Shi et al. 2026) evaluates knowledge-intensive tool use over 97 tasks, in which the LLM must retrieve and reason over unstructured documents. We report Reward mean, Pass^1–Pass^4, Action Recall, and Document Recall.

Experimental Setup

We keep Card construction strictly separate from held-out evaluation. Training trajectories are used to mine failure pat-

Setting	Model	Pass@1 (%)	Pass^5 (%)	UX
Baseline	Ministral-3-14B	30.50	8.70	2.95
	Qwen3-14B	24.00	7.30	2.90
	GPT-5.5	56.67	34.00	3.76
	Claude-Sonnet-5	62.00	36.67	4.07
	Claude-Opus-4-8	57.47	30.00	4.02
	DeepSeek-v4-Flash	56.90	30.00	3.72
	DeepSeek-v4-Pro	59.10	34.00	3.93
	MiMo-v2.5-Pro	54.50	27.30	3.96
	GPT-5.6-Sol	62.13	39.33	3.93
LoopBreaker	Ministral-3-14B	30.90 (**+0.40**)	11.30 (**+2.60**)	2.96 (+0.01)
	Qwen3-14B	26.80 (**+2.80**)	10.00 (**+2.70**)	2.90 (0)
	GPT-5.5	60.80 (**+4.13**)	35.33 (**+1.33**)	3.79 (+0.03)
	Claude-Sonnet-5	62.80 (**+0.80**)	36.00 (−0.67)	4.05 (−0.02)
	Claude-Opus-4-8	59.33 (**+1.86**)	30.67 (**+0.67**)	4.05 (+0.03)
	DeepSeek-v4-Flash	59.50 (**+2.60**)	34.00 (**+4.00**)	3.75 (+0.03)
	DeepSeek-v4-Pro	59.50 (**+0.40**)	34.00 (0)	3.94 (+0.01)
	MiMo-v2.5-Pro	56.30 (**+1.80**)	27.30 (0)	3.96 (0)
	GPT-5.6-Sol	62.67 (**+0.54**)	38.67 (−0.66)	3.96 (+0.03)

Table 2: Overall STATE-Bench test results. Each setting contains 150 tasks and five trials, with a fixed 750-trajectory Pass@1 denominator. Values in parentheses report LoopBreaker minus Baseline; positive gains are in **bold**. UX averages valid judged trajectories.

Setting	Model	Shopping Score (%)	Case Acc. (%)
Baseline	GPT-5.5	85.97	56.46
	GPT-5.6-Sol	85.54	54.79
	Claude-Sonnet-5	87.50	59.24
	Claude-Opus-4-8	87.23	61.25
	DeepSeek-v4-Flash	83.72	52.92
	DeepSeek-v4-Pro	82.33	51.67
LoopBreaker	GPT-5.5	86.49 (**+0.52**)	57.92 (**+1.46**)
	GPT-5.6-Sol	87.13 (**+1.59**)	59.58 (**+4.79**)
	Claude-Sonnet-5	89.32 (**+1.82**)	61.54 (**+2.30**)
	Claude-Opus-4-8	89.00 (**+1.77**)	62.85 (**+1.60**)
	DeepSeek-v4-Flash	84.81 (**+1.09**)	54.38 (**+1.46**)
	DeepSeek-v4-Pro	83.78 (**+1.45**)	55.63 (**+3.96**)

Table 3: Overall DeepPlanning Shopping test results. Each setting contains 120 tasks and four trials (480 task-runs). Values in parentheses report LoopBreaker minus Baseline; positive gains are in **bold**.

terns and instantiate Cards, and a development split is used to select the Registry. Every Card definition together with the controller's runtime settings is frozen before test execution.

We evaluate nine LLMs on STATE-Bench (five runs per task), six LLMs on DeepPlanning Shopping (four runs per task), and five model-version pairs on τ-Knowledge (four runs per task). For each LLM, Baseline uses the base model and LoopBreaker adds the same frozen benchmark-specific Registry, while tasks, tools, simulator, and evaluator are otherwise unchanged. Compilation is run independently per benchmark, so Cards are never pooled across environments.

Main Results

Capability on Interactive Service Tasks. On STATE-Bench, we test whether a selective recovery layer helps a broad range of tool-using LLMs across travel, customer-support, and shopping domains. As shown in Table 2, every paired comparison improves Pass@1, with the largest gain of $+4.13$ points on GPT-5.5. Pass^5 improves or holds for seven models, and UX is non-decreasing for eight. Because the strongest gains are not confined to the weakest or strongest baseline, the effect cannot be attributed to a single-model or ceiling explanation. The smaller movements in repeated-trial performance and UX further indicate that the benefit is concentrated in improved task completion. LoopBreaker therefore delivers consistent gains across nine LLMs spanning open-source and proprietary families.

Capability on Long-Horizon Shopping Planning. DeepPlanning Shopping stresses *long-horizon planning under verifiable constraints*, where each task involves multi-step product search and constraint satisfaction. Table 3 reports that LoopBreaker improves Shopping Score for every LLM, with the largest Case Accuracy gain of $+4.79$ points on GPT-5.6-Sol. Gains are consistent in direction and vary in magnitude across the six evaluated LLMs. This result shows that the LoopBreaker framework carries over to a longer and more constrained task horizon when a separate Registry is compiled.

Capability on Knowledge-Intensive Tool Use. τ-Knowledge targets *knowledge-intensive tool use*, in which the LLM must retrieve and reason over unstructured documents. As reported in Table 4, Reward improves for every matched model-version pair (up to $+5.36$ points on GPT-5.6-Sol), all Pass-rate differences remain non-negative, and Document Recall rises throughout. The same framework thus remains effective when a separate Registry is compiled for a knowledge-intensive environment beyond simulated services.

Taken together, the three tables show the same top-level pattern across different tools, task horizons, and model families: a selective recovery layer improves the primary task outcome without depending on a single benchmark or model. The remaining variation is concentrated in secondary measures, which is expected from a local intervention that changes only eligible portions of a trajectory.

Analyses

We now test whether the observed gains require compiled Cards, Registry validation, grounded arguments, and selective admission. Tables 5 and 7 evaluate full test trajectories, and Table 6 uses same-state forks.

Ablation Study on Compiled Recovery

To evaluate whether the gain requires compiled Cards, we replace Cards at either the functional-loop or exact-repeat trigger with a generic reconsideration prompt (Table 5). Neither variant improves on Baseline. The unvalidated all-Card Registry raises Pass@1 but intervenes far more often ($169/750$) with 15 observed losses, while the validated nine-Card Registry attains the best Pass@1 ($+2.6$) and UX with only one observed loss. This shows that LoopBreaker's validated Registry outperforms both generic reconsideration prompts and unvalidated Card sets, delivering larger gains with far fewer interventions. Because compiled Cards act deterministically, validation is done once and reused, and no new failure mode is introduced at inference time.

Setting	Model	Reward (%)	Pass^1 (%)	Pass^2 (%)	Pass^3 (%)	Pass^4 (%)	Action Recall (%)	Doc Recall (%)
Baseline	Claude-Sonnet-5	18.44	16.24	11.00	7.99	5.15	66.02	70.29
	Claude-Opus-4-8	18.29	17.27	10.65	7.22	5.15	48.12	62.15
	GPT-5.5	20.11	20.05	14.58	10.22	8.14	51.46	65.29
	DeepSeek-v4-Flash	20.10	20.10	12.37	9.79	8.25	72.97	70.35
	GPT-5.6-Sol	20.11	20.79	13.89	10.90	8.75	51.27	72.01
LoopBreaker	Claude-Sonnet-5	18.69 (**+0.25**)	17.01 (**+0.77**)	11.28 (**+0.28**)	8.19 (**+0.20**)	6.12 (**+0.97**)	66.70 (+0.68)	70.55 (+0.26)
	Claude-Opus-4-8	20.11 (**+1.82**)	20.10 (**+2.83**)	14.34 (**+3.69**)	11.66 (**+4.44**)	10.08 (**+4.93**)	51.22 (+3.10)	68.65 (+6.50)
	GPT-5.5	21.94 (**+1.83**)	22.16 (**+2.11**)	15.02 (**+0.44**)	11.74 (**+1.52**)	9.92 (**+1.78**)	52.24 (+0.78)	68.03 (+2.74)
	DeepSeek-v4-Flash	20.88 (**+0.77**)	20.88 (**+0.77**)	13.06 (**+0.69**)	10.05 (**+0.26**)	8.25 (0)	75.11 (+2.13)	71.35 (+1.00)
	GPT-5.6-Sol	25.46 (**+5.36**)	25.86 (**+5.07**)	18.40 (**+4.51**)	13.03 (**+2.13**)	10.00 (**+1.25**)	52.35 (+1.08)	72.89 (+0.88)

Table 4: Overall τ-Knowledge test results. Each setting contains 97 tasks and four trials (388 task-runs). Values in parentheses report LoopBreaker minus Baseline; positive gains on Reward and Pass^k are in **bold**.

Variant	Pass@1	Δ	Pass^5	Δ	UX	Δ	Triggered	W/T/L	Obs. Loss	Obs. Net
Baseline	56.9	–	30.0	–	3.72	–	0/750	–	–	–
Functional Loop + Generic Prompt	55.5	−1.4	28.0	−2.0	3.65	−0.07	51/750	0/50/1	2.0%	−2.0%
Exact Repeat + Generic Prompt	56.7	−0.3	30.7	+0.7	3.69	−0.04	102/750	6/87/9	8.8%	−2.9%
Unvalidated (All Cards)	58.5	+1.6	**36.0**	+6.0	3.74	+0.02	169/750	28/126/15	8.9%	+7.7%
Validated (9 Cards)	**59.5**	+2.6	34.0	+4.0	**3.75**	+0.03	45/750	13/31/1	2.2%	+26.7%

Table 5: Recovery and Registry ablation on STATE-Bench with DeepSeek-v4-Flash. Δ is relative to Baseline; triggered diagnostics use matched task-runs. Each variant contains 750 test trajectories. Bold marks the best value per column.

Recovery Branch	Success	Δ	W/T/L	Harm
No Intervention	9/49 (18.4%)	–	–	–
Full Card	24/49 (49.0%)	+30.6	16/32/1	2.0%
Operator-Only	9/49 (18.4%)	0.0	0/49/0	0.0%
No Arg. Grounding	16/49 (32.7%)	+14.3	7/42/0	0.0%

Table 6: Same-state Card-content ablation on 49 held-out pre-action states. Δ and diagnostics are relative to No Intervention. Bold marks the best value.

Controller	Pass@1	Pass^5	UX	Triggered	W/T/L	Harm
Full	**59.5**	**34.0**	**3.75**	45/750	13/31/1	2.2%
No-Guard	58.5	32.0	3.71	50/750	12/36/2	4.0%

Table 7: Controller-gate ablation on an independent 750-trajectory rerun. Triggered W/T/L and harm are diagnostics against Baseline. Bold marks the best value.

Effect of Grounded Card Content

To isolate the role of executable grounding, we compare Cards with different content on 49 held-out same-state forks (Table 6). The Full Card solves 24/49 states versus 9/49 without intervention (+30.6 absolute), removing argument grounding drops the count to 16/49, and an Operator-Only Card matches the no-intervention baseline. This confirms the superiority of LoopBreaker's typed programmatic Card design, in which executable arguments and constraints carry the recovery effect that a free-form operator label cannot. Unlike verbal reflection that asks the model to re-plan, each correction is bound to a compiled program and produces the same effect from the same state.

Effect of Controller Admission Gate

To isolate the admission gate, we remove the proposed-action and recent-progress guards on an independent 750-trajectory rerun (Table 7). Broader admission raises interventions from 45 to 50, and lowers Pass@1, Pass^5, and UX while doubling observed harm. This confirms that LoopBreaker's selective admission is essential to its gains, since it prevents mined Cards from harming productive retries. Broadening admission trades precision for coverage, and only the selective controller keeps both. Overall, these ablations highlight the joint necessity of Card validation and selective admission for LoopBreaker to reliably improve outcomes over generic reconsideration prompts.

Recovery Behavior on Triggered Trajectories

To measure LoopBreaker's impact where it is actually invoked, we focus on the subset of trajectories on which the controller intervened. Figure 6 reports the net outcome margin per base model. Across 140 triggered trajectories, LoopBreaker produces 39 wins and 9 losses, giving 30 net gains, and the direction is positive for every base model. This demonstrates that LoopBreaker's frozen Registry transfers across LLM families without per-model tuning, and the uniform direction suggests these patterns describe failure modes that recur across model architectures.

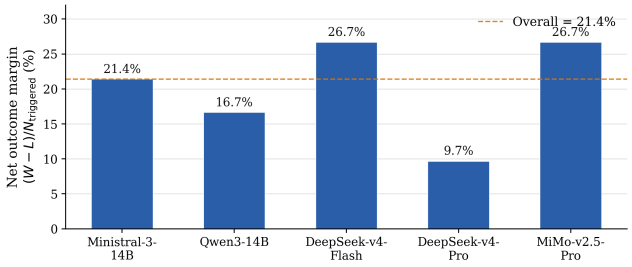


Figure 6: Net outcome margin among triggered STATE-Bench trajectories is positive for every base model, and beneficial interventions outnumber harmful ones on the triggered subset by $(39 - 9)/140 = 21.4\%$ overall.

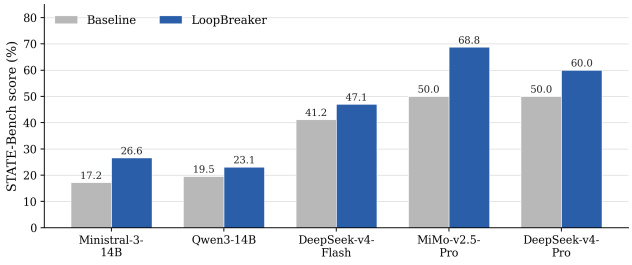


Figure 7: On task-runs whose baseline trajectory contains a functional loop before intervention, LoopBreaker raises the STATE-Bench score for every base model.

Recovery Behavior on Functional Loop-Exposed Trajectories

To test whether the gain comes from repairing functional loops themselves, we isolate task-runs whose baseline trajectory already contains a functional loop before intervention. Figures 7 and 8 show that scores rise for all five base models while tool calls fall in every case, with the largest score gains on MiMo-v2.5-Pro and DeepSeek-v4-Pro. This validates that LoopBreaker precisely targets the failure mode it is designed for, and raising scores while cutting tool calls rules out gains from simply spending more interaction budget. Overall, these subset results highlight LoopBreaker's deployment behavior on Functional Loop-exposed trajectories, while the same-state experiment in Table 6 provides the corresponding controlled evidence for the recovery program itself.

Related Work

Recent work approaches tool-use failures from several angles. Early-exit methods reduce ineffective behavior by deciding when to halt a stalled trajectory (Lu et al. 2025b), while termination alone does not repair a state that a small correction could still resolve. Reflection frameworks such as Reflexion (Shinn et al. 2023) and Self-Refine (Madaan et al. 2023) accumulate verbal feedback across trials, and CRITIC (Gou et al. 2024) augments this self-verification with external tools. Test-time replanning generates a fresh recovery plan through another generation step, as in Reasoning Inception (Kim et al. 2026), Reinforced Agent (Ta, Zhu, and Shayandeh 2026), and LATS (Zhou et al. 2024), while the correction is only as reliable as the underlying model. Workflow and skill memory approaches (Wang et al. 2025b,a) reuse successful trajectories as general-purpose skills, and policy optimization methods learn recovery from execution errors through reinforcement learning (Zhang et al. 2026c; Kang et al. 2026; Zhang et al. 2026a). Overall, these methods either halt too early, invoke another generation, or require gradient access, and none of them treats the Functional Loop as a distinct recoverable execution state addressed by a validated local program.

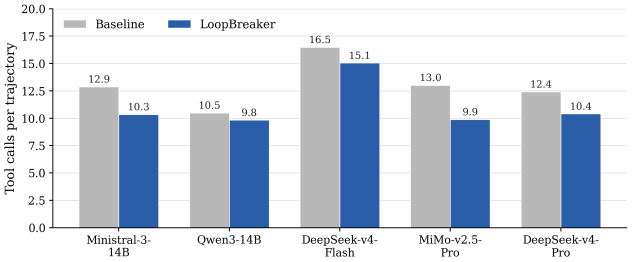


Figure 8: On the same Functional Loop-exposed task-runs, LoopBreaker reduces the average number of tool calls per trajectory for every base model.

In this work, we focus on inference-time recovery from a well-defined execution failure, with the goal of escaping a Functional Loop through a grounded local correction while leaving the base LLM's policy unchanged. Unlike previous approaches that either halt a stalled trajectory, generate an ungrounded replan, or update the model via reinforcement learning, LoopBreaker adopts a more deliberate design by pairing an offline experience compiler with an online selective controller that admits at most one executable Card once a Functional Loop is detected.

Conclusion

In this paper, we introduced the Functional Loop as a distinct failure mode in tool-using LLMs, in which feedback-visible actions make no task progress and repeat the same unresolved effect. Our analysis across STATE-Bench, DeepPlanning Shopping, and τ-Knowledge shows that Functional Loops are prevalent, are associated with lower task performance, and are not well captured by string-level tool repetition. Along with this observation, we proposed LoopBreaker, a local recovery layer that reuses concrete training-time failures at inference time and admits at most one grounded Recovery Card once a Functional Loop is detected. Across three tool-use benchmarks and nine LLMs, LoopBreaker consistently improves the primary task outcome, and controlled ablations locate the gain in validated Cards under selective admission. LoopBreaker offers a step toward reusing concrete failure experience during tool execution, and complements existing efforts on tool selection, replanning, and policy optimization.

References

Barres, V.; Dong, H.; Ray, S.; Si, X.; and Narasimhan, K. 2026. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. In *International Conference on Machine Learning*.

Chen, C.; Hao, X.; Liu, W.; Huang, X.; Zeng, X.; Yu, S.; Li, D.; Huang, Y.; Liu, X.; Wang, X.; and Liu, W. 2025. ACEBench: A Comprehensive Evaluation of LLM Tool Usage. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 12970–12998. Association for Computational Linguistics.

Gou, Z.; Shao, Z.; Gong, Y.; Shen, Y.; Yang, Y.; Duan, N.; and Chen, W. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*.

He, W.; Sun, Y.; Hao, H.; Xia, Z.; Hao, X.; Gu, Q.; Su, H.; and Cai, X. 2026. VitaBench: Benchmarking LLM Agents with Versatile Interactive Tasks in Real-world Applications. In *International Conference on Learning Representations*.

Kang, H.; Suresh, T.; Saad-Falcon, J.; and Mirhoseini, A. 2026. TRACE: Capability-Targeted Agentic Training. *arXiv preprint arXiv:2604.05336*.

Kim, T.; Nam, J.; Basu, C.; Fan, X.; Ma, C.; Ji, H.; Tur, G.; and Hakkani-Tür, D. 2026. ReIn: Conversational Error Recovery with Reasoning Inception. In *International Conference on Learning Representations*.

Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; Wang, Z.; and Pang, R. 2025a. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1160–1183. Association for Computational Linguistics.

Lu, Q.; Ding, L.; Cao, S.; Liu, X.; Zhang, K.; Zhang, J.; and Tao, D. 2025b. Runaway is Ashamed, But Helpful: On the Early-Exit Behavior of Large Language Model-based Agents in Embodied Environments. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 24014–24027. Association for Computational Linguistics.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, volume 36.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerstein, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, volume 36.

Shi, Q.; Zytek, A.; Razavi, P.; Narasimhan, K. R.; and Barres, V. 2026. τ-Knowledge: Evaluating Conversational Agents over Unstructured Knowledge. In *International Conference on Machine Learning*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 36.

Ta, A.; Zhu, J.; and Shayandeh, S. 2026. Reinforced Agent: Inference-Time Feedback for Tool-Calling Agents. *arXiv preprint arXiv:2604.27233*.

Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Dong, V.; Li, E.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 16022–16076. Association for Computational Linguistics.

Wang, Z. Z.; Gandhi, A.; Neubig, G.; and Fried, D. 2025a. Inducing Programmatic Skills for Agentic Tasks. In *Conference on Language Modeling*.

Wang, Z. Z.; Mao, J.; Fried, D.; and Neubig, G. 2025b. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 63897–63911. PMLR.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Zhang, T.; Popa, A.-I.; Xu, Y.; Song, R.; and Dimitriadis, D. 2026a. PIVOT: Bridging Planning and Execution in LLM Agents via Trajectory Refinement. *arXiv preprint arXiv:2605.11225*.

Zhang, Y.; Jiang, S.; Li, R.; Tu, J.; Su, Y.; Deng, L.; Guo, X.; Lv, C.; and Lin, J. 2026b. DeepPlanning: Benchmarking Long-Horizon Agentic Planning with Verifiable Constraints. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7377–7407. Association for Computational Linguistics.

Zhang, Z.; Zhao, F.; Wang, R.; Wang, Z.; Liang, B.; Wang, J.; Hu, Y.; Cao, S.; and Wong, K.-F. 2026c. Robust Tool Use via Fission-GRPO: Learning to Recover from Execution Errors. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 40477–40491. Association for Computational Linguistics.

Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; and Wang, Y.-X. 2024. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR.